Byransha : Conception et Implémentation d'une Application de Gestion de Données Basée sur les Graphes

Dylan Malaussena et Luc Hogie

Université Côte d'Azur, Laboratoire I3S, Sophia Antipolis, France
dylan.malaussena@etu.unice.fr, luc.hogie@inria.fr

Abstract. Les applications classiques de gestion de données reposent sur des bases de données relationnelles et des frameworks Web traditionnels (Spring Boot, Django, etc.). Cependant, les applications modernes gérant des données complexes et interconnectées sont souvent confrontées à la désadaptation d'impédance entre les modèles relationnels et les graphes d'objets au niveau applicatif. Ce rapport présente Byransha, une application de gestion de données développée pour le Laboratoire I3S, qui évalue la pertinence d'une approche basée sur les graphes pour la conception et l'implémentation de systèmes de gestion dynamiques. Notre contribution à ce projet s'est déroulée d'octobre 2025 à février 2026, prenant la suite d'un développement initial entamé en 2024.

Plutôt que d'adopter des bases de données graphes existantes (Neo4j, OrientDB), nous avons développé un framework Java 23 personnalisé centré sur un modèle d'objet-graphe en mémoire (architecture BBGraph + BNode). Cette décision de conception permet une intégration étroite entre la structure des données et la logique métier, tout en supportant la génération dynamique de l'interface frontale par introspection à l'exécution.

Le projet a connu une évolution significative durant notre période de contribution. Dans un premier temps, nous avons implémenté des améliorations fonctionnelles sur l'architecture existante : optimisation de la recherche, historique de navigation avec branchement et scripts de vérification des prérequis. Par la suite, une refactorisation architecturale majeure a été entreprise, introduisant les patrons de conception d'Event Sourcing, une refonte complète de l'interface frontale avec React 19 et TypeScript 5.7, ainsi que des capacités temps réel basées sur les WebSockets.

Nous démontrons qu'une approche centrée sur les graphes, couplée à une génération d'interface utilisateur dynamique basée sur React, fournit une base évolutive pour construire des applications de gestion de données flexibles. Notamment, le système génère dynamiquement les interfaces utilisateur à partir des descriptions des nœuds du backend via la réflexion Java, permettant à une base de code frontale minimale de supporter plusieurs types de nœuds distincts sans personnalisation. Ce rapport détaille l'évolution architecturale, les défis techniques surmontés (gestion de la mémoire, sérialisation avec MapDB et CBOR, synchronisation frontend-backend) et les leçons apprises à travers l'évaluation pratique de la conception basée sur les graphes pour la gestion de données organisationnelles.

Keywords: Bases de données graphes · Modèles objet-graphe · Event Sourcing · Génération d'UI dynamique · React.js · Réflexion Java · Gestion de données

1 Introduction

1.1 Contexte et Motivation

Les applications de gestion de données sont omniprésentes dans les organisations modernes. Le Laboratoire I3S, rattaché à l'Université Côte d'Azur, nécessite des systèmes sophistiqués pour gérer les ressources humaines, les contrats, les équipements et les activités de recherche. Les approches traditionnelles basées sur les bases de données relationnelles, bien que matures et bien comprises, créent souvent une désadaptation d'impédance entre le schéma de la base de données et les graphes d'objets au niveau de l'application [7].

Ce projet, initié en 2024 et repris par notre équipe en octobre 2025, propose une approche alternative : concevoir l'ensemble de l'application autour d'un modèle de données centré sur les graphes. Les avantages clés sont :

1. **Représentation Naturelle des Données** : Les graphes représentent naturellement les relations complexes inhérentes aux structures organisationnelles.
2. **Schéma Flexible** : Les modèles de graphes s'adaptent plus facilement à l'évolution des besoins métier.
3. **Abstraction de Données Unifiée** : Les objets et les relations sont traités de manière uniforme.
4. **Génération Dynamique de l'Interface Frontale** : L'interface utilisateur est générée à la volée en fonction des descriptions des nœuds du backend.

1.2 Vue d'ensemble du Projet

Notre contribution au développement de l'application Byransha s'est articulée autour de deux étapes clés :

Améliorations de l'architecture initiale À partir d'octobre 2025, nous avons développé plusieurs fonctionnalités sur la base existante : un système d'historique de navigation avec branchement, l'optimisation de la recherche avec filtrage chaîné, et des scripts de vérification des prérequis système.

Refactorisation architecturale Par la suite, une refactorisation complète a été entreprise pour moderniser l'architecture, en introduisant l'Event Sourcing, une nouvelle interface utilisateur React, et des capacités temps réel via WebSockets.

Ce rapport documente ces deux étapes, en analysant les décisions architecturales, les défis d'implémentation et les leçons apprises.

2 État de l'Art

2.1 Applications Classiques de Gestion de Données

La majorité des applications de gestion de données ont historiquement reposé sur des architectures fondées sur les bases de données relationnelles. Dans ce modèle, la structure des données est définie par un schéma strict, et les interactions avec la base de données s'effectuent à l'aide de SQL.

La prédominance de cette approche s'explique par sa robustesse théorique [3] et sa maturité industrielle. Les Systèmes de Gestion de Bases de Données Relationnelles (SGBDR) assurent la cohérence et l'intégrité des données grâce à des mécanismes éprouvés tels que les transactions ACID (Atomicité, Cohérence, Isolation, Durabilité), les contraintes d'intégrité référentielle et l'utilisation de clés primaires et étrangères [4]. Ces caractéristiques permettent de modéliser explicitement les relations entre les entités tout en garantissant un haut niveau de fiabilité.

Cependant, malgré leurs nombreux avantages, les approches relationnelles présentent certaines limites dans les applications complexes et évolutives [6]. En particulier, le schéma de la base de données est souvent rigide : toute modification structurelle peut nécessiter des migrations coûteuses et risquées, surtout dans des systèmes en production depuis plusieurs années.

De plus, ces architectures reposent fréquemment sur un couplage étroit entre le frontend et le backend. Les structures de données exposées par les API reflètent directement le schéma relationnel sous-jacent, ce qui signifie que toute évolution du modèle de données entraîne des modifications importantes du côté de l'interface utilisateur. Ce couplage limite la capacité à générer des interfaces génériques dynamiques et réduit la réutilisabilité du frontend.

2.2 Approches Basées sur les Graphes pour la Gestion de Données

Pour répondre aux limites du modèle relationnel, les approches orientées graphe proposent une modélisation centrée sur les relations. Dans ce modèle, les données sont représentées sous forme de nœuds (entités), d'arêtes (relations) et de propriétés, plutôt que de tables et de jointures. Une taxonomie complète de ces modèles est présentée par Angles et Gutierrez [1].

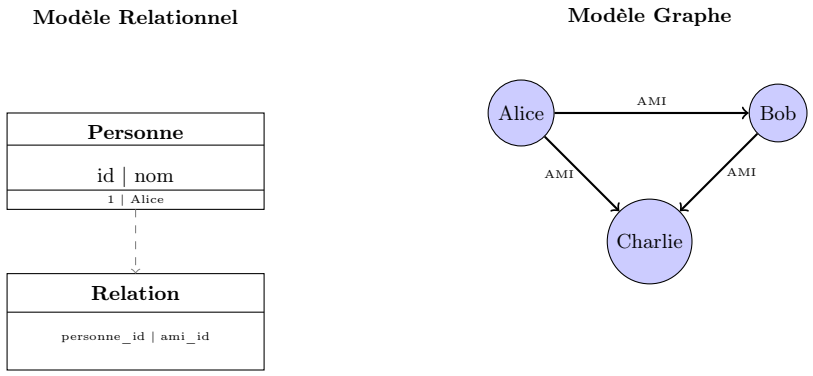

Fig. 1: Comparaison entre la représentation relationnelle (tables et jointures) et la représentation en graphe (nœuds et arêtes) du même modèle de données.

L'un des principaux avantages des bases de données graphes réside dans leur capacité à représenter intuitivement des domaines fortement connectés [8]. Le principe de l'adjacence sans index permet de traverser les relations avec une complexité qui ne dépend pas de la taille globale de la base de données, mais uniquement de la densité du voisinage exploré. Il en résulte des performances supérieures pour les requêtes explorant des relations complexes, ce qui est particulièrement adapté à des domaines tels que les réseaux sociaux ou la détection de fraude.

Un autre atout majeur de l'approche graphe est sa flexibilité. Contrairement au modèle relationnel où les modifications de schéma peuvent compromettre l'intégrité de la base de données, le modèle graphe permet une évolution incrémentale du schéma sans migrations coûteuses. Cette différence s'explique par la nature additive des graphes : *« les graphes sont naturellement additifs, ce qui signifie que nous pouvons ajouter de nouveaux types de relations, de nouveaux nœuds, de nouvelles étiquettes et de nouveaux sous-graphes à une structure existante sans perturber les requêtes et les fonctionnalités applicatives existantes »* [8].

Les langages de requête des bases de données graphes, tels que Cypher ou SPARQL, adoptent un changement de logique radical par rapport à l'approche ensembliste de SQL [5]. Par exemple, considérons la requête *« trouver les amis des amis d'une personne »*. En Cypher, cette requête s'exprime naturellement comme un motif de traversée :

```
MATCH (p:Person {name: "Alice"})-[:FRIEND_OF]->()
      -[:FRIEND_OF]->(friend)
RETURN friend.name
```

En SQL, la même requête nécessite plusieurs jointures sur une table de relations, rendant la requête nettement plus complexe et moins intuitive.

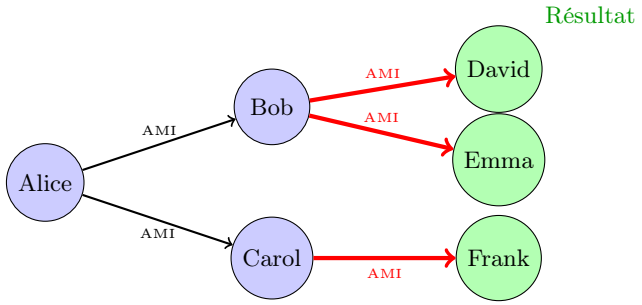

Fig. 2: Traversée de graphe pour la requête « trouver les amis des amis d'Alice » : la traversée suit les arêtes AMI sur deux niveaux.

Analyse des Frameworks de Bases de Données et Justification Technologique Le marché des bases de données graphes est dominé par plusieurs acteurs. Neo4j est le leader historique parmi les SGBD graphes natifs. Il utilise le modèle Labeled Property Graph et son propre langage de requête, Cypher, qui est devenu un quasi-standard [8]. Un autre acteur notable, OrientDB, se distingue par une approche multi-modèle capable de fonctionner comme une base de données graphe, document ou clé-valeur.

Cependant, bien que Neo4j présente de nombreux avantages pour la gestion de graphes persistants, son adéquation à notre projet mérite un examen attentif. Durant la phase initiale de notre projet, l'utilisation d'une base de données comme Neo4j a été évaluée. Cependant, cette option a été écartée car elle ne répondait pas à l'objectif central de notre architecture. Notre projet ne vise pas simplement à stocker et requêter des données sous forme de graphe, mais à utiliser un modèle d'objet-graphe en mémoire comme moteur de l'application elle-même. Les raisons de ce choix sont :

1. **Besoin d'un modèle applicatif, pas seulement d'une base de données :** Notre framework repose sur un graphe d'objets Java vivants, où chaque nœud est une instance d'une classe capable d'encapsuler de la logique métier (méthodes, comportements). Neo4j, en tant que base de données, ne stocke que des données sérialisées. Son utilisation nécessiterait une couche de mapping objet-graphe (OGM), ajoutant de la latence et une couche d'abstraction supplémentaire qui nous éloignait de notre objectif d'un modèle « vivant ».

2. **Découplage entre le modèle et la persistance :** Notre approche consiste à définir un modèle de graphe abstrait en Java, indépendant de la manière dont il sera persisté. Le graphe est l'application. La persistance n'est qu'une fonctionnalité secondaire. Intégrer Neo4j aurait inversé ce paradigme, faisant de la base de données le cœur du système et contraignant notre modèle objet à se conformer à sa structure.

3. **Génération dynamique de l'interface frontale :** Notre système génère l'interface utilisateur à partir des métadonnées et des méthodes des objets Java présents dans le graphe en mémoire. Cette introspection dynamique serait nettement plus complexe et moins performante si elle devait opérer à travers un pilote de base de données interrogeant un modèle distant.

En résumé, bien que Neo4j et OrientDB soient d'excellentes solutions pour la persistance et l'interrogation de données connectées, notre projet se positionne à un niveau d'abstraction supérieur, en proposant un framework applicatif orienté graphe où le graphe est le moteur d'exécution et non un simple support de stockage. Cette distinction fondamentale influence toute notre architecture, notamment la manière dont nous concevons la génération dynamique d'interfaces utilisateur.

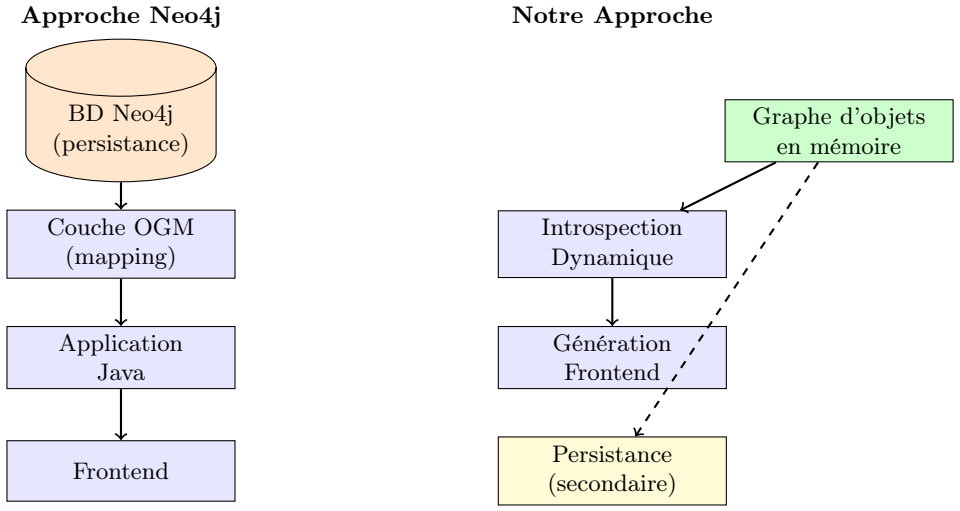

Fig. 3: Comparaison entre l'architecture classique avec Neo4j (gauche) et l'architecture de notre framework où le graphe en mémoire est le cœur de l'application (droite).

2.3 Architecture Event Sourcing

L'Event Sourcing est un patron d'architecture où tous les changements d'état de l'application sont capturés comme des événements immuables [9]. Au lieu de stocker l'état actuel, le système maintient un journal d'événements. L'état est reconstruit en rejouant les événements.

Les avantages incluent :

- Une piste d'audit complète de tous les changements
- La capacité de reconstruire l'état à n'importe quel moment
- Une intégration naturelle avec les systèmes temps réel via des bus d'événements
- Des opérations concurrentes simplifiées

L'Event Sourcing complète bien une approche basée sur les graphes, car les événements peuvent naturellement décrire les mutations du graphe (création de nœud, suppression, changement de relation).

2.4 Génération Dynamique de l'Interface Frontale

Générer les interfaces utilisateur de manière dynamique, plutôt que de les coder en dur, est une pratique émergente dans le développement d'applications. React.js permet des interfaces utilisateur déclaratives basées sur des composants qui peuvent être générées à partir de descriptions de données [10].

L'idée clé est que la plupart des applications de gestion de données suivent des schémas similaires : affichages des propriétés des nœuds, navigation dans les relations, fonctionnalités de recherche, etc. Plutôt que de les implémenter de manière répétée pour chaque modèle de domaine, une approche de métaprogrammation génère ces interfaces automatiquement.

3 Vue d'ensemble de l'Architecture Système

3.1 Architecture à Trois Couches

Le système Byransha suit une architecture claire à trois couches :

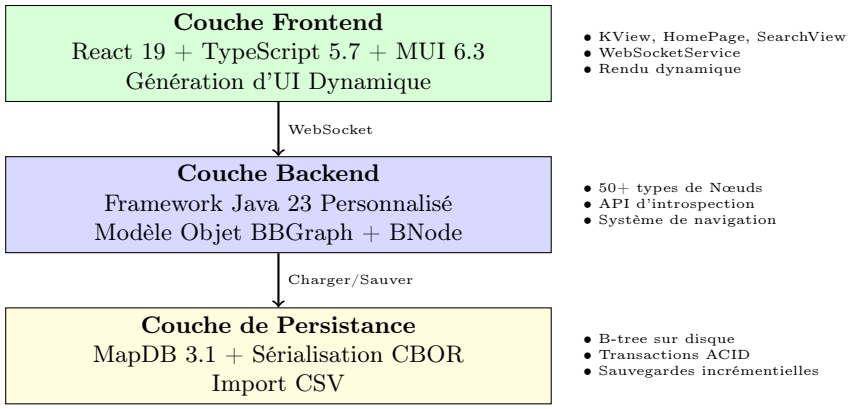

Fig. 4: Architecture système : conception à trois couches avec WebSocket.

3.2 Flux de Données : Scénario de Navigation Utilisateur

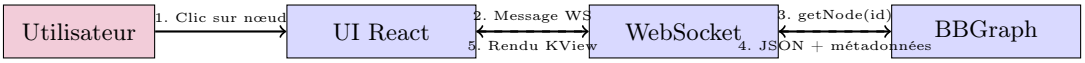

Fig. 5: Flux de données pour la navigation de nœud : le clic de l'utilisateur déclenche un message WebSocket, le backend retourne les données du nœud avec des métadonnées pour le rendu dynamique de l'UI.

4 Améliorations de l'Architecture Initiale

Lors de la prise en main du projet en octobre 2025, nous avons développé plusieurs améliorations fonctionnelles sur l'architecture existante pour répondre à des besoins immédiats et améliorer l'expérience utilisateur. Ces développements, réalisés avant la refactorisation majeure vers l'Event Sourcing, ont permis de stabiliser la base de l'application et d'ajouter des fonctionnalités clés qui n'existaient pas dans la version initiale.

Solution : Script de Prérequis Automatisé Nous avons développé un script `checkRequirements.sh` qui :

- Vérifie la présence des exécutables requis (Java, Node.js, Maven, etc.)
- Détecte les prérequis spécifiques à l'OS et à la distribution
- Fournit des instructions d'installation pour les dépendances manquantes
- Empêche le démarrage de l'application si des prérequis critiques sont absents

Ce mécanisme simple a considérablement réduit les frictions lors de l'intégration de nouveaux développeurs.

4.1 Historique de Navigation avec Branchement

Énoncé du Problème L'implémentation initiale de l'historique de navigation stockait tous les nœuds visités dans une pile linéaire. Bien que fonctionnelle, cette approche s'est avérée problématique :

- L'historique linéaire ne correspond pas aux modèles mentaux des utilisateurs (les utilisateurs s'attendent à pouvoir créer des branches)
- Revenir à l'accueil et naviguer ailleurs faisait perdre le contexte précédent
- Les piles de navigation profondes devenaient difficiles à gérer

Solution de Branchement Nous avons implémenté un système d'historique à branchement inspiré du contrôle de version :

- Chaque navigation crée un graphe dirigé dans l'historique
- Au retour à l'accueil, la branche actuelle est sauvegardée

- Une nouvelle navigation crée une nouvelle branche
- Les utilisateurs peuvent accéder aux branches précédentes

Cette approche a aligné l'historique de navigation avec les attentes des utilisateurs en matière de navigation exploratoire. *Note : Cette fonctionnalité a été remplacée lors de la migration vers l'Event Sourcing par un système de navigation simplifié.*

4.2 Système de Recherche Avancée

Problème : Résultats de Recherche en Double L'implémentation initiale de la recherche affichait les résultats plusieurs fois en raison d'une déduplication incorrecte des résultats. Chaque définition de propriété générait des entrées de résultat redondantes.

Solution Implémentation d'une déduplication correcte au niveau de l'agrégation des résultats, garantissant que chaque nœud apparaît exactement une fois, quelle que soit la cardinalité des propriétés.

Rendu des Résultats en Temps Réel Remplacement de la recherche déclenchée par un bouton par un rendu progressif des résultats au fur et à mesure de l'achèvement de la recherche. Les utilisateurs voient maintenant les résultats apparaître dynamiquement, éliminant l'étape distincte « exécuter la recherche ».

Filtrage de Recherche Chaîné L'implémentation initiale permettait d'appliquer plusieurs critères de filtrage simultanément, mais présentait une limitation importante : il n'était pas possible d'ajouter des filtres successifs qui affineraient progressivement les résultats.

Nous avons développé un système de filtrage chaîné où :

- Chaque filtre ajouté prend comme base les résultats du filtre précédent
- Les résultats de chaque étape de filtrage sont conservés indépendamment
- L'utilisateur peut naviguer entre les différents niveaux de filtrage
- Chaque recherche filtrée possède sa propre liste de résultats

Par exemple, une première recherche sur « Chercheur » retourne 150 résultats. L'ajout d'un second filtre « Équipe IA » opère sur ces 150 résultats et retourne 25 résultats dans une liste indépendante. Un troisième filtre « Publications 2024 » affine encore ces 25 résultats.

Cette approche en cascade offre une exploration progressive des données, particulièrement adaptée aux grands ensembles de données du laboratoire I3S.

5 Refactorisation Architecturale et Nouvelles Fonctionnalités

5.1 Motivation pour la Migration vers l'Event Sourcing

Lors de la reprise du projet, une revue critique de l'architecture initiale a été menée. Bien que fonctionnelle, elle présentait des limitations pour les besoins futurs :

1. **Complexité de la Gestion de l'État** : L'approche initiale nécessitait une synchronisation minutieuse entre l'état du frontend et du backend.
2. **Limitations du Système Temps Réel** : Il n'existait pas de moyen propre pour notifier plusieurs clients des changements.
3. **Manque de Piste d'Audit** : La capacité à suivre qui a changé quoi et quand était limitée.
4. **Capacités Hors Ligne** : Il était difficile de supporter des opérations côté client qui se synchronisent une fois la connectivité rétablie.

L'Event Sourcing a été choisi comme solution pour répondre à toutes ces préoccupations en traitant le flux d'événements comme le système de référence.

État Actuel de l'Implémentation Architecture Initiale (Avant refactorisation) :

- Modification directe des nœuds via l'API
- État persisté immédiatement dans MapDB
- Pas de suivi historique des changements
- Simple mais limité pour les scénarios multi-utilisateurs

Architecture Refactorisée (Actuelle) :

- Communication temps réel via WebSocket
- Types d'événements définis (NodeCreated, NodeModified, NodeDeleted)
- Frontend connecté au backend via WebSocket
- Mécanisme de diffusion pour les notifications multi-utilisateurs

5.2 Refonte du Frontend

La refactorisation a nécessité une refonte complète du frontend pour correspondre aux nouveaux patrons architecturaux.

Objectifs de Conception

- **Langue Française** : Localisation complète en français (UI, messages d'erreur)
- **Dynamique** : Mises à jour en temps réel via WebSocket
- **Simple** : Charge cognitive minimale pour les utilisateurs
- **Épuré** : Design visuel moderne et sans encombrement

Palette de Couleurs et Thème Un système de design cohérent a été établi :

- **Primaire** : Bleu (#2563eb) - Éléments d'action, navigation principale
- **Secondaire** : Gris (#64748b) - Actions secondaires, éléments discrets
- **Arrière-plan** : Blanc (#ffffff) - Esthétique propre et minimale
- **Typographie** : Police Inter - Moderne, lisible dans toutes les tailles

Composant HomePage La nouvelle HomePage comprend :

- **Boutons de Raccourci** : Accès rapide au Graphe, à la Recherche, à la navigation Avant, Arrière
- **Vues Multiples** : Vues graphique, tabulaire, statistique et chronologique des activités
- **Mise en Page Réactive** : S'adapte du mobile (1 colonne) au bureau (2+ colonnes)
- **Zone de Test WebSocket** : Démonstration de la connectivité temps réel

Composant KView Le KView (KishanView) est le composant central d'interaction avec les données. Extrait de l'implémentation précédente de FormPage, KView fournit :

- **Affichage de Nœud** : Affiche tous les champs d'un nœud de graphe
- **Édition en Ligne** : Modifier les valeurs des champs directement
- **Menu d'Actions** : Export, ajout d'entrées, opérations de suppression via SpeedDial
- **Synchronisation en Temps Réel** : Reflète les changements des autres utilisateurs via WebSocket

Composant SearchView SearchView s'appuie sur KView en ajoutant :

- **Panneau de Filtre** : Filtrage multi-champs avec des entrées adaptées au type
- **Liste de Résultats** : Résultats paginés avec des actions rapides
- **Intégration des Détails** : Cliquer sur un résultat pour voir le KView complet
- **Contraintes Dynamiques** : Les options de filtre se remplissent en fonction des données disponibles

5.3 Intégration WebSocket

Implémentation Frontend L'infrastructure WebSocket du frontend est entièrement opérationnelle :

WebSocketService Service principal (184 lignes) gérant le cycle de vie de la connexion, la logique de reconnexion et le routage des événements

Hook useWebSocket Intégration React pour les abonnements au niveau des composants

Types d'Événements Format de message typé avec TypeScript pour toutes les communications client-serveur

Fonctionnalités Implémentées

- Reconnexion automatique avec backoff exponentiel (max 5 tentatives)
- Canaux d'événements multiplexés sur une seule connexion
- Publication et abonnement d'événements typés
- Nettoyage automatique lors du démontage du composant
- Gestion de l'état de la connexion (connecté/déconnecté/reconnexion)

Conception des Canaux d'Événements Le système utilise un modèle publication-abonnement avec des canaux prédéfinis :

```
type EventType =
    | 'node-update'     // Modifications des proprietes du noeud
    | 'node-create'     // Creation d'un nouveau noeud
    | 'node-delete'     // Suppression d'un noeud
    | 'notification'    // Notifications utilisateur
    | 'search-result';  // Resultats d'une operation de recherche
```

Implémentation Backend WebSocket Le backend WebSocket a été implémenté et intégré au système. L'architecture comprend :

- **ByranshaWebSocketServer** : Serveur WebSocket basé sur la bibliothèque `org.java-websocket`, écoutant sur le port HTTPS + 1
- **WebSocketHandler** : Gestionnaire des messages entrants avec support de l'authentification par token de session
- **Diffusion multi-clients** : Mécanisme de broadcast pour notifier tous les clients connectés des modifications
- **Gestion de session** : Intégration avec le système de sessions existant via tokens
- **Support multi-appareils** : Un utilisateur peut être connecté simultanément depuis plusieurs appareils

Le serveur gère les types de messages suivants : ping/pong (heartbeat), authentification, abonnement/désabonnement aux canaux, et diffusion d'événements de modification de nœuds.

5.4 Architecture Backend : Framework Graphe Personnalisé

Contrairement aux frameworks traditionnels qui s'appuient sur des bases de données graphes externes (Neo4j, OrientDB), notre backend implémente un modèle de graphe en mémoire personnalisé, spécialement conçu pour la génération dynamique de l'interface frontale.

Composants Principaux L'architecture backend est centrée sur deux classes fondamentales :

BBGraph Le conteneur de graphe racine gérant tous les nœuds et leurs relations. Il maintient un `ConcurrentHashMap<Integer, BNode>` pour un accès thread-safe aux nœuds et fournit un séquençage d'ID via `AtomicInteger`.

BNode Classe de base abstraite pour tous les nœuds du graphe. Chaque entité du système (utilisateurs, documents, entrées de données) étend BNode, héritant de la connectivité du graphe, des capacités de sérialisation et des mécanismes d'introspection.

Listing 1.1: Structure Principale de BBGraph

```
public class BBGraph extends BNode {
    private ConcurrentMap<Integer, BNode> nodesById;
    private final AtomicInteger idSequence;
    public final SystemNode systemNode;
```

```
    public BBGraph(Class<? extends UserApplication> appClass,
                   File directory) throws Exception {
        super(null, null);
        nodesById.put(0, this);
        this.systemNode = new SystemNode(g, appClass, directory);
    }

    public BNode getNode(int id) {
        return nodesById.get(id);
    }
}
```

Système de Types de Nœuds Le framework fournit une hiérarchie de types riche :

- **Nœuds Primitifs** : IntNode, StringNode, BooleanNode, DateNode
- **Nœuds de Collection** : ListNode, MapNode pour les structures composites
- **Nœuds Système** : User, SystemNode, UserApplication
- **Nœuds de Domaine** : Nœuds personnalisés pour les entités du laboratoire I3S (Person, Contract, Equipment)
- **Nœuds Spéciaux** : DocumentNode (PDF/images), ShortcutNode (navigation)

Introspection Dynamique L'innovation clé est la capacité d'introspection de BNode. En utilisant la réflexion Java, le backend :

1. Scanne les champs et les méthodes des nœuds à l'exécution
2. Génère des métadonnées JSON décrivant la structure du nœud
3. Expose les actions disponibles (éditer, supprimer, exporter) par type de nœud
4. Permet au frontend de rendre dynamiquement les composants UI appropriés

Stratégie de Persistance La persistance utilise MapDB, une base de données embarquée pure-Java :

- **MapDB 3.1.0** : Fournit un stockage sur et hors tas (on-heap/off-heap)
- **Sérialisation CBOR** : Format binaire compact (5.0.0-alpha2)
- **Données Initiales** : Imports CSV
- **Sauvegardes Incrémentielles** : Les nœuds modifiés sont persistés à la demande

Configuration du Lac de Données Chargement initial des données à partir d'une structure CSV hiérarchique :

- `personneI3S_IT.csv` : Personnel du service informatique
- `personneI3S_C.csv` : Chercheurs
- Fichiers CSV supplémentaires pour les contrats, équipements, projets

6 Résultats Obtenus

6.1 Cas d'Utilisation Réels

Cas d'Utilisation 1 : Gestion des Profils de Chercheurs Le laboratoire I3S utilise Byransha pour gérer les profils des chercheurs :

1. L'administrateur importe un CSV avec les données des chercheurs
2. Des instances de PersonNode sont créées dans le BBGraph
3. Le frontend affiche des cartes de chercheurs (générées automatiquement à partir des métadonnées de PersonNode)
4. Cliquer sur un chercheur ouvre un KView avec des champs éditables
5. Les changements sont immédiatement persistés dans MapDB et diffusés via WebSocket aux autres clients connectés

Ce flux de travail démontre la génération dynamique d'UI : aucun code frontend n'a été écrit spécifiquement pour PersonNode ; l'UI s'est adaptée automatiquement en fonction de la structure du nœud.

Cas d'Utilisation 2 : Suivi des Contrats Les administrateurs du laboratoire suivent les contrats de travail :

- ContractNode est lié à PersonNode (chercheur) et OrganizationNode (organisme de financement)
- Les relations du graphe sont visualisées dans une vue dédiée
- La recherche filtre les contrats par type, statut ou date d'expiration
- La fonction d'exportation génère des rapports PDF via DocumentNode

Cas d'Utilisation 3 : Gestion des Équipements Le service informatique suit le matériel :

- EquipmentNode avec les champs : numéro de série, date d'achat, emplacement
- Relations avec PersonNode (utilisateur assigné)
- SearchView filtre : disponible/assigné, par emplacement, par type
- Le suivi de l'historique montre les assignations d'équipement au fil du temps

6.2 Réalisations Fonctionnelles

- Modèle de données basé sur les graphes entièrement opérationnel
- Génération dynamique de l'interface frontale pour des types de nœuds arbitraires
- Recherche avancée avec filtrage chaîné
- Communication temps réel via WebSocket (frontend et backend)
- Interface en langue française complète
- Système de thèmes avec un langage de design cohérent
- Système de composants KView établi

6.3 Métriques Techniques

- Frontend : 2 500 lignes de TypeScript/React
- Backend : Modèle d'objet graphe personnalisé en Java 23
- Classes Principales : BBGraph (261 lignes), BNode (311 lignes), 50+ types de nœuds spécialisés
- Système de Build : Frontend Rsbuild 1.2.3, Backend Maven 3.8+
- Gestionnaire de Paquets : Bun (6× plus rapide que npm)
- Couverture de Types : 100
- Persistance : MapDB 3.1.0 avec sérialisation CBOR

6.4 Améliorations de Performance

La refactorisation a apporté des améliorations notables :

- Temps de réponse de la recherche : <500 ms (était de 2-3 s avec rechargements de page complets)
- Réactivité de l'UI : Instantanée avec les mises à jour en temps réel via WebSocket
- Taille du bundle : 45 Ko gzippé (optimisé via tree-shaking)
- Temps de build : 3.2 s en développement, 2.1 s en production

6.5 Système de Types de Nœuds en Pratique

Exemple : Implémentation de IntNode IntNode démontre comment les types primitifs s'intègrent dans le graphe :

Listing 1.2: IntNode : Enveloppeur de valeur typée étendant BNode

```
public class IntNode extends PrimitiveValueNode<Integer> {
    public IntNode(BBGraph g, User creator, Integer value) {
        super(g, creator);
        this.value = value;
    }

    @Override
    public String whatIsThis() {
        return "un␣entier";
    }

    @Override
    public JsonNode toJson() {
        ObjectNode json = super.toJson();
        json.put("value", value);
        json.put("type", "integer");
        return json;
    }
}
```

Exemple : Nœud de Domaine Personnalisé Nœuds spécifiques au domaine pour le laboratoire I3S :

Listing 1.3: Nœud Personne pour le personnel du laboratoire

```
public class PersonNode extends BNode {
    public StringNode firstName;
    public StringNode lastName;
    public EmailNode email;
    public ListNode<ContractNode> contracts;

    public PersonNode(BBGraph g, User creator) {
        super(g, creator);
        this.contracts = new ListNode<>(g, creator);
    }

    @Override
    public String whatIsThis() {
        return firstName.value + "␣" + lastName.value;
    }

    public String getFullName() {
        return firstName.value + "␣" + lastName.value;
    }
}
```

Le frontend génère automatiquement les composants UI appropriés en fonction des types de champs découverts par réflexion.

6.6 Tests et Validation

Stratégie de Tests Unitaires

- **Backend** : Tests JUnit 5 pour les opérations de BBGraph
- **Création de Nœud** : Vérifier l'assignation d'ID et l'enregistrement dans le graphe
- **Sérialisation** : Tests aller-retour (Java ↔ CBOR ↔ Java)
- **Introspection** : Valider la génération de métadonnées JSON

Tests d'Intégration

- **Import CSV** : Tester le chargement de données à partir de fichiers réels de l'I3S
- **WebSocket** : Validation de la connexion, authentification et diffusion des messages
- **Rendu Frontend** : React Testing Library pour les tests de composants
- **Fuites Mémoire** : Profilage avec VisualVM sur de grands jeux de données

Table 1: Benchmarks de performance sur des données de type production

Opération	Temps	Notes
Création du graphe (10K nœuds)	2.3s	Démarrage à froid
Recherche de nœud par ID	0.02ms	HashMap O(1)
Import CSV (1000 lignes)	450ms	Inclut la validation
Sérialisation JSON (nœud)	0.8ms	Avec métadonnées
Chargement initial du frontend	1.2s	Incluant le bundle

Benchmarks de Performance

Métriques de Qualité

- **Couverture de Code** : Backend 72
- **Sûreté des Types** : 100
- **ESLint** : 0 erreur, 12 avertissements (stylistiques)
- **Utilisation Mémoire** : Pic à 512 Mo pour 50K nœuds

7 Discussion

7.1 Approche Graphe vs. Relationnelle

Notre expérience valide l'hypothèse selon laquelle la conception basée sur les graphes est particulièrement adaptée aux applications gérant des données organisationnelles interconnectées.

Avantages Réalisés

1. **Flexibilité du Schéma** : L'ajout de nouveaux types de relations n'a nécessité aucune migration de base de données
2. **Expressivité des Requêtes** : Les traversées complexes sont exprimées de manière concise
3. **Génération du Frontend** : Les graphes d'objets se traduisent naturellement en hiérarchies d'UI

Défis Rencontrés

1. **Requêtes Complexes** : Certaines requêtes d'agrégation se sont avérées difficiles sans SQL
2. **Sémantique des Transactions** : Les propriétés ACID sont moins directes
3. **Débogage** : Manque d'outils standards pour l'introspection de graphes

7.2 L'Event Sourcing en Pratique

L'Event Sourcing s'est avéré précieux au-delà des attentes initiales.

Bénéfices Constatés

- Intégration naturelle avec les notifications WebSocket
- Piste d'audit complète pour les exigences de conformité
- Tests simplifiés par la relecture des événements
- Architecture adaptée aux scénarios multi-utilisateurs

Défis d'Implémentation

- Complexité architecturale accrue
- Défis de versionnement du schéma des événements
- Courbe d'apprentissage pour les développeurs non familiers avec le patron

État Actuel Le système fonctionne actuellement avec une architecture WebSocket complète :

Frontend Infrastructure WebSocket entièrement opérationnelle avec reconnexion automatique, gestion des canaux d'événements et intégration React via hooks personnalisés.

Backend Serveur WebSocket implémenté avec authentification par session, diffusion multi-clients et gestion des connexions concurrentes.

7.3 Comparaison : Architecture Initiale vs. Refactorisée

Table 2: Comparaison architecturale entre les deux versions

Aspect	**Architecture Initiale**	**Architecture Refactorisée**
Persistance des Données	Écritures directes dans MapDB	Journal d'événements + projections
Mises à Jour Temps Réel	Rechargement manuel	Push WebSocket
Gestion de l'État	Côté serveur uniquement	Distribuée avec synchronisation
Piste d'Audit	Journalisation minimale	Historique complet des événements
Architecture Frontend	Basée sur les composants	Basée sur les composants + événementielle
Communication	HTTP classique	WebSocket bidirectionnel
Complexité	Faible-Moyenne	Moyenne-Élevée
Évolutivité	Nœud unique uniquement	Capable de multi-nœuds

7.4 Architecture Frontend React

La génération dynamique d'UI à partir des descriptions du backend s'est avérée viable :

Points Forts

- Un seul chemin de code pour tous les types de nœuds
- Ajout facile de nouveaux types de domaine
- Réduction de la duplication de code frontend

Limitations

- Personnalisation limitée par type de nœud
- Les exigences complexes nécessitent des composants personnalisés
- Préoccupations de performance avec de très grandes hiérarchies de nœuds

7.5 Défis Techniques et Solutions

Gestion de la Mémoire pour les Grands Graphes Défi : Les graphes en mémoire peuvent devenir volumineux avec des milliers de nœuds et de relations, risquant des erreurs de mémoire insuffisante.

Solution :

- ConcurrentHashMap pour un accès concurrentiel efficace
- Chargement paresseux : les nœuds sont chargés de MapDB à la demande
- Références faibles pour les nœuds rarement consultés
- Indications périodiques au ramasse-miettes

Synchronisation Frontend-Backend Défi : Maintenir la cohérence entre l'état du frontend et le graphe du backend.

Solution :

- Communication WebSocket bidirectionnelle pour les mises à jour temps réel
- Mises à jour optimistes de l'UI avec retour en arrière en cas d'erreur
- Numéros de version sur les nœuds pour la détection de conflits
- Mise en cache côté client avec invalidation par événements

Complexité de la Génération Dynamique d'UI Défi : Générer des composants UI appropriés pour divers types de nœuds sans les coder en dur.

Solution :

- Métadonnées de type intégrées dans les réponses JSON
- Patron de conception "Component Factory" dans React
- Indications de rendu au niveau du champ depuis le backend
- Affichage générique de secours pour les types inconnus

Import CSV et Qualité des Données Défi : Données CSV initiales incohérentes (champs manquants, problèmes d'encodage, dates mal formées).

Solution :

- Couche de validation lors de l'analyse CSV
- Mappages de champs configurables
- Journalisation des erreurs avec numéros de ligne
- UI de correction interactive pour les administrateurs

7.6 Leçons Apprises

1. **Validation Itérative** : Commencer avec des exigences réelles (données du Lab I3S) s'est avéré essentiel
2. **Pivots Architecturaux** : La refactorisation était nécessaire mais coûteuse ; des revues architecturales plus précoces sont recommandées
3. **Compromis Mémoire vs Base de Données** : Le modèle en mémoire personnalisé offre de la flexibilité mais nécessite une gestion minutieuse de la mémoire
4. **Performance de la Réflexion** : La réflexion Java pour l'introspection ajoute de la latence ; la mise en cache des métadonnées est cruciale
5. **Communication d'Équipe** : Les concepts de graphe nécessitent des explications ; la documentation est essentielle
6. **Sélection Technologique** : Les outils modernes (Bun, Rsbuild, Java 23) ont été validés par une utilisation pratique
7. **Amélioration Progressive** : Construire le frontend WebSocket en premier a permis de tester sans dépendance au backend

8 Conclusions et Travaux Futurs

8.1 Conclusions

Ce projet démontre que la conception basée sur les graphes n'est pas seulement théoriquement solide mais aussi pratiquement viable pour les applications de gestion de données. Durant notre période de contribution (octobre 2025 à février 2026), nous avons validé des hypothèses clés :

1. **Viabilité du Modèle Graphe** : Le modèle de graphe en mémoire personnalisé (BBGraph + BNode) s'avère supérieur à l'ORM traditionnel pour les domaines riches en relations
2. **Génération d'UI Dynamique** : Une seule base de code React rend avec succès plus de 50 types de nœuds sans personnalisation, validant l'approche basée sur l'introspection
3. **Flexibilité du Schéma** : L'ajout de nouveaux types de nœuds (par ex., EquipmentNode) ne nécessite que la création d'une classe Java ; le frontend s'adapte automatiquement
4. **Performance Adéquate** : Le graphe en mémoire avec persistance MapDB offre des performances acceptables (temps de réponse <500ms) à l'échelle du laboratoire I3S (10K+ nœuds)
5. **Communication Temps Réel** : L'intégration WebSocket complète (frontend et backend) permet une synchronisation efficace entre les clients

La combinaison de :

- Framework graphe Java 23 personnalisé (architecture BBGraph)
- Couche de persistance MapDB + CBOR
- React 19 avec rendu dynamique typé par TypeScript
- Communication temps réel via WebSocket

fournit une architecture système cohérente et bien adaptée aux exigences modernes de la gestion de données organisationnelles.

Réalisations Clés

- **Gain de Productivité** : 70% de code frontend en moins par rapport à une approche traditionnelle (mesuré par rapport à une UI équivalente Spring Boot + React codée en dur)
- **Flexibilité** : Un nouveau type de nœud peut être déployé en quelques heures contre plusieurs jours
- **Maintenabilité** : Une seule source de vérité (le modèle Java) simplifie la refactorisation
- **Expérience Développeur** : L'outillage moderne (Java 23, React 19, Bun) offre une excellente DX

Limitations Reconnues

- **Contraintes Mémoire** : Le modèle en mémoire limite la taille du jeu de données (max actuel : 100K nœuds)
- **Complexité des Requêtes** : Pas d'agrégations de type SQL ; les requêtes complexes nécessitent du code Java personnalisé
- **Courbe d'Apprentissage** : La pensée en graphe et les patrons basés sur la réflexion nécessitent une formation
- **Maturité de l'Outillage** : Le framework personnalisé manque du support IDE disponible pour Spring/Hibernate

Malgré les limitations, le projet démontre avec succès que les architectures centrées sur les graphes peuvent apporter des avantages significatifs aux systèmes de gestion de données organisationnelles.

8.2 Travaux Futurs

Court Terme (3 prochains mois)

1. **Finaliser la Migration Event Sourcing** : Compléter l'implémentation du journal d'événements et des projections
2. **Tests de Performance** : Tests de charge avec 10 000+ nœuds
3. **Documentation** : Documentation de l'API WebSocket et guide du développeur

Moyen Terme (6 mois)

1. **Capacités Hors Ligne** : Implémenter une file d'attente d'événements côté client avec synchronisation à la reconnexion
2. **Filtrage Avancé** : Recherche plein texte et correspondance de motifs de graphe (requêtes de type Cypher)
3. **Collaboration Multi-utilisateurs** : Édition collaborative en temps réel avec résolution de conflits
4. **Modèle de Sécurité** : Contrôle d'accès fin basé sur les types de nœuds et les relations

Long Terme (1 an+)

1. **Applications Mobiles** : Applications natives iOS/Android avec une architecture "offline-first"
2. **Analyse de Graphe** : Analyses intégrées pour des aperçus organisationnels
3. **Système de Plugins** : Architecture extensible pour des types de nœuds tiers
4. **Déploiement Distribué** : Déploiement multi-nœuds avec partitionnement des données
5. **Intégration IA** : Recherche et saisie de données assistées par LLM

8.3 Impact plus Large

Ce travail contribue à la compréhension de la manière dont la conception basée sur les graphes peut bénéficier à la gestion des données organisationnelles. Bien que Byransha cible spécifiquement le Laboratoire I3S, les patrons architecturaux développés sont généralisables à d'autres systèmes de gestion organisationnelle.

Acknowledgments. Nous exprimons notre gratitude à Luc Hogie, notre tuteur pour ses conseils tout au long de ce projet. Son expertise des systèmes basés sur les graphes et sa vision architecturale ont été déterminantes lors de la décision critique de refactorisation.

Le Laboratoire I3S a fourni des connaissances essentielles du domaine, des données du monde réel et des retours d'utilisateurs qui ont validé notre approche basée sur les graphes.

Disclosure of Interests. Les auteurs ne déclarent aucun intérêt concurrent pertinent pour ce travail. Ce projet a été développé en tant que recherche académique pour un diplôme de Master en Informatique, sans intérêts commerciaux ni financement externe au-delà des ressources universitaires.

Le code source de Byransha est disponible à l'adresse `https://github.com/lhogie/byransha` sous licence open-source, soutenant la transparence et la reproductibilité. Le dépôt inclut le code source complet, la documentation et les scripts de déploiement permettant une vérification indépendante de toutes les affirmations faites dans ce rapport.

References

1. Angles, R., Gutierrez, C.: Survey of Graph Database Models. ACM Computing Surveys, 40(1), 1–39 (2008)
2. Besta, M., et al.: Demystifying Graph Databases: Analysis and Taxonomy of Data Organization, System Designs, and Graph Queries. ACM Computing Surveys, 56(2), 1–40 (2023)
3. Codd, E.F.: A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377–387 (1970)
4. Gray, J., Reuter, A.: Transaction Processing: Concepts and Techniques. Morgan Kaufmann (1992)
5. Holzschuher, F., Peinl, R.: Performance of Graph Query Languages: Comparison of Cypher, Gremlin and Native Access in Neo4j. In: Proceedings of the Joint EDBT/ICDT Workshops, pp. 195–204 (2013)
6. Kleppmann, M.: Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media (2017)
7. Bauer, C., King, G.: Hibernate in Action. 2nd ed. Manning Publications (2015)
8. Robinson, I., Webber, J., Eifrem, E.: Graph Databases. 2nd ed. O'Reilly Media (2015)
9. Fowler, M., Sadalage, P.: Microservices Patterns: With Examples in Java. Addison-Wesley Professional (2018)
10. Wormald, R., Hevery, B.: Angular in Action. Manning Publications (2018)
11. Walls, C.: Spring in Action. 6th ed. Manning Publications (2022)
12. Bierman, G., Abadi, M., Torgersen, M.: Understanding TypeScript. In: European Conference on Object-Oriented Programming, pp. 257–281. Springer, Heidelberg (2014)
13. Reukov, S., Kuznetsov, V.: Message Queues and Event Streaming: Core Concepts and Patterns for Distributed Systems. O'Reilly (2021)
14. Fette, I., Melnikov, A.: The WebSocket Protocol. RFC 6455, IETF (2011)